

Private Network Service Platform — Project Guide

Team roles: **Mac 1** = DNS server, **Mac 2** = Edge/Reverse Proxy/LB + TLS, **Mac 3** = Backend A, **Mac 4** = Backend B (+ test client). Domain: `app.teamX.test` / `api.teamX.test`. Use ports 8080/8443 if 80/443 are restricted.

PHASE 1 — Build & Observe

Task A — Establish the Private LAN

1. Connect all Macs to the same Wi-Fi/hotspot/LAN.
2. On each Mac, record: private IPv4 (`ipconfig getifaddr en0`), subnet mask, default gateway, interface name, MAC address.
3. `ping` every Mac from every other Mac — confirm full reachability.
4. Draw a topology diagram showing all 4 roles and their connections.

Task B — Configure a Private DNS Server (Mac 1)

1. Install dnsmasq: `brew install dnsmasq`.
2. Add DNS records:

```
app.teamX.test → <Mac 2 IP>
api.teamX.test → <Mac 2 IP>
```

3. Start dnsmasq: `sudo brew services start dnsmasq`.
4. On at least 2 other Macs: System Settings → Network → DNS → set Mac 1's IP as resolver.
5. Verify: `dig app.teamX.test` (or `nslookup`) from a client — should return Mac 2's IP.
6. Access the app only by domain name, never by IP.

Task C — Build Two Simple Backend Services (Mac 3 & Mac 4)

1. Write a minimal REST API (Node/Express or Python/Flask) on each machine.
2. Required endpoints:
 - `GET /` → basic confirmation page/JSON.
 - `GET /api/status` → `{ "backend": "A", "status": "ok" }` (or "B").
3. Every response must include header `X-Backend: A` (or `B`).
4. Bind to the LAN interface (e.g., `0.0.0.0`), **not** `127.0.0.1`.
5. Fixed ports: Backend A = 3001, Backend B = 3002.

Task D — Configure the Edge Reverse Proxy and Load Balancer (Mac 2)

1. Install nginx: `brew install nginx`.
2. Configure an upstream block with both backends (Mac 3:3001, Mac 4:3002).
3. Use round-robin (default) or `least_conn`.
4. Reload nginx and test: run repeated `curl` requests to the domain, confirm `X-Backend` alternates between A and B.
5. Be ready to explain why clients never need backend IPs directly.

Task E — Add HTTPS / TLS

1. Generate a cert for `app.teamX.test` — use `mkcert` (easiest, auto-trusted) or OpenSSL self-signed.
2. Configure nginx to terminate TLS on port 443/8443, proxying to the backends.
3. Install the CA certificate on every client Mac's trust store so there are no browser/curl warnings.
4. Final demo must use plain `curl https://...` — no `-k` flag.
5. Be ready to explain the handshake: ClientHello → ServerHello → Certificate → Key Exchange → Finished.

Task F — Demonstrate HTTP Caching Behavior

1. Add `Cache-Control: max-age=60` header on at least one endpoint.
2. Optionally add an `ETag`.
3. Show headers: `curl -I <url>`.
4. Demonstrate either a cache hit (browser dev tools Network tab) or a `304 Not Modified` conditional request.
5. Be ready to explain: fresh hit vs. conditional request vs. full new request.

Task G — Capture the Complete Protocol Flow

1. Run Wireshark on the client Mac during a fresh request cycle.
2. Capture and be ready to point to:
 - DNS query + response (port 53/UDP).
 - TCP three-way handshake: SYN → SYN-ACK → ACK (note source/dest ports).
 - TLS handshake: ClientHello, ServerHello, Certificate, ChangeCipherSpec.
 - HTTP request/response headers (`curl -v`) or browser dev tools).
 - Alternating `X-Backend` header across repeated requests.
3. Save the `.pcapng` file as evidence.

Required Phase 1 Failure Demonstrations

Deliberately trigger each and explain what happens and why:

1. **Wrong DNS server on client** → name lookup fails, but direct IP connectivity still works (DNS ≠ IP layer).
2. **DNS record points to wrong IP** → resolution succeeds, client reaches wrong destination (DNS is a directory, not a connection).
3. **One backend stopped** → edge keeps serving via the remaining backend (if health-check/failover configured).
4. **Both backends stopped** → DNS/TLS still work, but app returns `502 Bad Gateway` (shows edge vs. backend boundary).
5. **Wrong destination port on client** → host reachable, but TCP connection to the service fails (ports ≠ IP addresses).

Phase 1 gate: client resolves `app.teamX.test`, connects over HTTPS, and gets responses from both backends via the load balancer.

PHASE 2 — Harden, Recover, and Troubleshoot

(Extends Phase 1 infrastructure — no rebuild.)

Extension A — Backup DNS Resolver

1. Run a second dnsmasq instance on a Mac other than Mac 1, with identical DNS records.
2. Configure clients to list both primary (Mac 1) and backup DNS servers.
3. Stop dnsmasq on Mac 1 — show clients still resolve via the backup.
4. Explain: DNS service failure (resolution stops) vs. application server failure (resolution works, service down).

Extension B — DNS TTL and Controlled Record Change

1. Set a short TTL (e.g., 30s) on a DNS record.
2. Resolve the name from a client, then change the record to a different IP.
3. Observe the client still getting the old (cached) answer until TTL expires, then the new one.
4. Demonstrate manually flushing the DNS cache to see the immediate change.
5. Explain the relation to DNS-based traffic cutover in production.

Extension C — Service Isolation (Backend Firewall Rules)

1. Restrict direct access to ports 3001/3002 so only Mac 2 (edge) can reach them — use macOS firewall/`pf` rules.

2. Demonstrate: Mac 2 can still reach backends; a client Mac cannot connect directly to 3001/3002.
3. Keep a rollback copy of the rules and restore original config after demo.

Extension D — High-Availability Failover Behavior

1. Configure nginx to detect backend unavailability (health checks).
2. Stop Backend A (Mac 3) — show all requests still served via Backend B.
3. Restart Backend A — show load balancing resumes across both.
4. Identify the current single point of failure (edge nginx on Mac 2) and explain what would eliminate it.

Extension E — Controlled Edge Migration (DNS-Based Cutover)

1. Configure a standby nginx on a spare Mac with the same config as Mac 2.
2. Update the DNS record for `app.teamX.test` to point to the standby edge.
3. Observe TTL effects: some clients briefly still hit the old edge while new lookups go to standby.

Extension F — Faculty-Injected Troubleshooting Challenge

1. Faculty introduces a configuration fault during evaluation.
2. Diagnose systematically and out loud: DNS → TCP → TLS → Application, in that order.
3. Identify and isolate/fix the affected layer. Partial credit is given for correct methodology even without a full fix.

Phase 2 gate: backup DNS, TTL behavior, service isolation, HA failover, and at least one troubleshooting challenge are all demonstrated.

Deliverables

Deliverable	Contents
Architecture Document	Topology diagram, machine roles + IP/service table, request-flow diagram showing each protocol layer (updated after Phase 2)
Configuration Bundle	dnsmasq config, nginx configs (Phase 1 + 2), TLS certificate setup notes, backend launch instructions, firewall rules
Backend Source Code	Full code for both backends + helper scripts (GitHub repo or zip)
Evidence Folder	DNS resolution screenshots (dig/nslookup), curl/browser output with headers, Wireshark captures (DNS, TCP handshake, TLS handshake), HTTP caching demo, all failure-scenario demos
Phase 1 Checkpoint	Working live system + structured demo; every member ready to explain any component
Phase 2 Final Report	What changed from Phase 1, resilience test results, one paragraph per extension
Final Presentation	Live demo following the 11-step sequence below; every member participates and answers individual viva questions

Steps to Reproduce (Live Demo Sequence)

Run through these 11 steps in order during evaluation:

1. **Show topology and inventory** — display network diagram, IP table, service map.
2. **Confirm LAN connectivity** — `(ping)` between all machines, show IPs.
3. **Resolve the private domain** — `(dig app.teamX.test)` from a client, confirm it resolves to Mac 2's IP via your team DNS.
4. **Open the service over HTTPS** — browse/curl `(https://app.teamX.test)`, no cert warnings, no IP in the URL.
5. **Show load balancing** — repeated `(curl)` requests, alternating `(X-Backend: A)` / `(X-Backend: B)`.
6. **Show Wireshark evidence** — open saved capture, point to DNS query/response, TCP handshake, TLS handshake.
7. **Show caching behavior** — `(curl -I)` showing `(Cache-Control)`, demonstrate a `(304)` or cache hit.

8. **Fail one backend** — stop Mac 3, show requests still succeed via Mac 4 only.
9. **Show Phase 2 resilience** — backup DNS failover, TTL/cutover, or DNS-based migration (pick one or more of A/B/E).
10. **Diagnose the injected fault** — faculty breaks something; diagnose layer by layer live.
11. **Individual viva** — every team member answers questions from personal understanding, no notes.